its legacy rival — monolithic architecture. Unlike a microservice, a monolith application is always built as a single, autonomous unit. In a two-tier architecture model with client and server components running, the server-side application handles web requests and executes programming logic to handle the data fetching and updating in the underlying database.

Though monolithic architecture is simple in style, the key problem is that all modules are tied up together and hence a change in one module can potentially affect the other. For example, a load in the backend service can affect the response time of GUI. Any small modification made to a piece of code in the application requires building and deploying the complete application. This is where creating a microservice helps.

Microservices architecture optimises resource usage and enables agility in development, where individual teams develop their own microservice which is integrated through API calls or asynchronous messages for inter-process communication. Most importantly, build and deployment is independent of each microservice, which makes it most suitable for continuous delivery.

Netflix, eBay, Amazon, Twitter, PayPal, Gilt, Bluemix, Soundcloud and many other large-scale websites and applications have all evolved from monolithic to microservices architecture.

API pattern development with microservices

Endpoint protection and load balancing requests are very important in the design of a microservices architecture during cloud adoption. There are two ways to handle this efficiently – direct-client-to-microservices pattern and API gateway pattern.

Direct-client-to-microservices pattern is a lightweight solution generally used for small- to medium-sized microservices architecture where server-side web applications can connect to microservices directly without any broker or bridge interface. The performance of such requests is superfast. However, in a production environment where clustered microservice instances are deployed, we need to manually introduce an application gateway to handle the load balancer and manually handle SSL termination in the endpoint instance itself (which can be resource intensive when there are more requests). When a secured request is transmitted through SSL/HTTPS, SSL termination is the process of offloading the data traffic to decrypt and send to the endpoint.

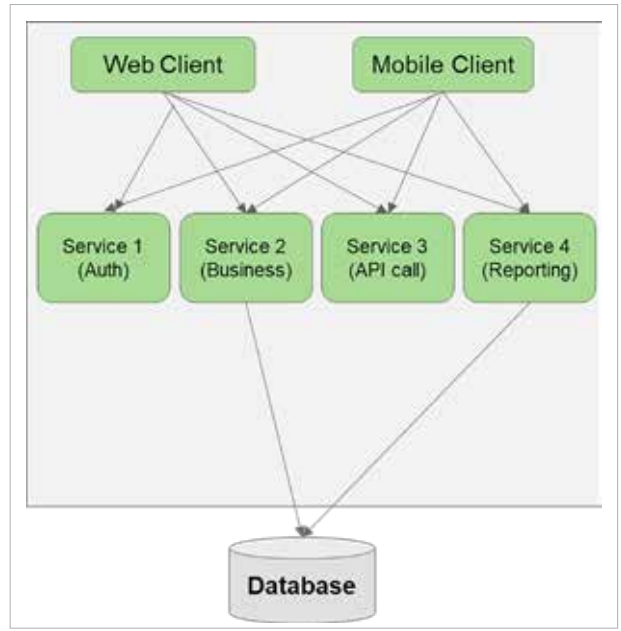

Figure 1: Example of microservices architecture

An API gateway pattern is adaptive and flexible, and introduces an API gateway between consumers and microservices. This avoids coupling of microservices requests with multiple microservices, round trips in requests, security issues like request filtering and role-based access and authorisation, and SSL offloading. This pattern is also called 'backend to frontend pattern' as it acts as a reverse-proxy in handling requests from clients to services. It can also be used to cache requests and serve future requests quickly for better response handling.

For large enterprise solutions, multiple API gateway facilities are optimal. For example, one API gateway to handle web consumers (single page applications) and one for mobile consumers to transform requests as HTML or JSON objects. An API gateway can also act as a request aggregator, handling multiple requests by collecting them and transmitting to single client requests. API gateway is available in Azure, AWS and GCP, and facilitates service discovery, IP filtering requests and logging/tracking requests or API service management, among other things.

A platform for microservices assessment

If you want to assess APIs and microservices for architectural flexibility, pattern-based microservices design, and understand their risks, there is an easy to use online assessment platform available at *https://microservices.io/platform/microservice-architecture-assessment.html*. This was designed by Chris Richardson, the author of the book 'Microservices Patterns'.

This survey-based qualitative assessment platform collects data about your microservices such as base architecture, development velocity if we have fully utilised the microservice construct in design, development infrastructure, organisational process in handling these services with respect to deployment/outage, and individual